

Enterprise Knowledge Copilot with Action Execution

A 3-Week Hands-On Blueprint (MCP + RAG + LangGraph + pgvector)

1. Enterprise Problem Statement & Architecture

The use case

"DevOps Copilot" — an internal chat assistant for a mid-size engineering org that:

1. **Answers questions** by retrieving from a corpus of internal docs (runbooks, architecture decision records, incident postmortems) using RAG over `pgvector`.
2. **Takes action**, not just answers — via MCP tools it can (a) query a live "incident" Postgres table, (b) create a Jira-style ticket (mocked), (c) post a Slack-style notification (mocked). This is the part that separates it from a toy RAG chatbot: it's an *agent* that decides when to retrieve vs. when to act, using LangGraph to manage that decision state explicitly rather than a single prompt-and-hope loop.
3. **Streams** its reasoning and tool calls to the frontend token-by-token, including intermediate "tool call" events, not just a final answer — this is what makes the Playwright testing interesting (asserting against an async, multi-event UI).

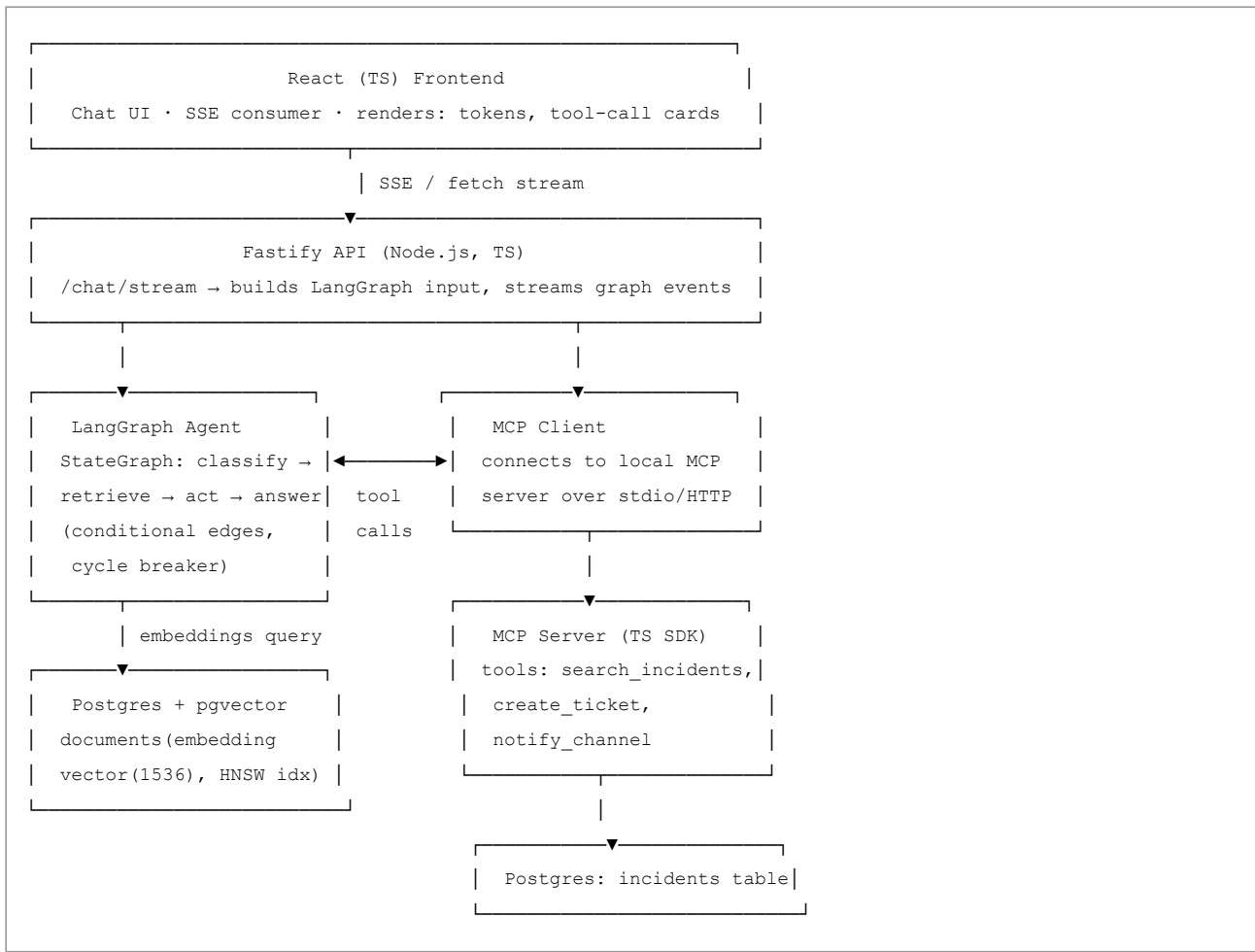
This is deliberately scoped to be *demoable in 10 minutes* but touch every skill on your list. It's realistic: "internal copilot that can look things up and also do things" is one of the most common enterprise GenAI patterns in 2025–2026 (paired with the shift toward MCP as a standardized tool-calling layer instead of bespoke REST integrations per agent).

Request lifecycle (Mermaid)

```
sequenceDiagram
    participant U as User (React UI)
    participant API as Fastify API (SSE stream)
    participant G as LangGraph Agent
    participant V as pgvector (RAG)
    participant M as MCP Server (Tools)
    participant DB as Postgres (Incidents)

    U->>API: POST /chat/stream {message, threadId}
    API->>G: invoke(graph, state, streamMode="values")
    G->>G: Node: classifyIntent (retrieve vs act vs both)
    alt Needs knowledge
        G->>V: similarity_search(embedding, k=5)
        V-->>G: top-k chunks + scores
    end
    alt Needs action
        G->>M: tools/call {name: "create_incident_ticket", args}
        M-->>DB: INSERT INTO incidents ...
        DB-->>M: row
        M-->>G: tool_result
    end
    G->>G: Node: synthesizeAnswer (stream tokens)
    G->>API: stream: token / tool_call / tool_result / done
    API-->>U: SSE events (rendered incrementally)
```

High-level component diagram



2. Zero-Friction Setup Guide

How this section works from here on: before every new tool or concept, there's a ☐ **Concept primer** written for someone who hasn't touched this in a decade — no assumed jargon. Then the code/command, then a **line-by-line breakdown** of literally every line, then, where relevant, **alternatives & why not**. Don't skim the primers — the commands only make sense once the primer's mental model is in place.

2.1 Postgres + pgvector via Docker

☐ Concept primer: what is Docker, and why are we using it at all?

Normally, "installing Postgres" means downloading it, running an installer, configuring it for your specific OS, and hoping it doesn't conflict with some other database already on your machine. A **container** is a way to skip all of that: it's a self-contained, pre-packaged little Linux environment that already has Postgres (and, here, the pgvector extension) installed and configured correctly, ready to run identically on any machine. **Docker** is the tool that runs containers. A `docker-compose.yml` file is just a recipe that says "here's one or more containers I want, here's how to configure them, here's how they should talk to each other" — instead of typing a long `docker run ...` command by hand every time, you write it once and `run docker compose up`.

One more term: an **image** is the packaged template (e.g., "Postgres 16 with pgvector already built in"); a **container** is a running instance of that image. You can start/stop/delete containers freely without re-downloading the image each time.

`docker-compose.yml` (repo root) — line by line

```

version: "3.9"

services:
  postgres:
    image: pgvector/pgvector:pg16
    container_name: copilot-postgres
    restart: unless-stopped
    environment:
      POSTGRES_USER: copilot
      POSTGRES_PASSWORD: copilot_dev_pw
      POSTGRES_DB: copilot_db
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U copilot -d copilot_db"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  pgdata:

```

Line	What it means
version: "3.9"	The version of the Compose file <i>format</i> itself (not Docker's version) — tells Docker Compose which set of features/syntax to expect in this file. 3.9 is a recent, widely-supported version.
services:	The list of containers this file manages. We have exactly one, named <code>postgres</code> (that name is just a label we chose — it's how other config sections in this file refer to it).
image: pgvector/pgvector:pg16	Which pre-built image to download and run. <code>pgvector/pgvector</code> is the image name (published by the pgvector project), and <code>pg16</code> is the tag — the specific version, here "built on Postgres 16." Alternative & why not: you could use the plain official <code>postgres:16</code> image and manually compile the pgvector extension into it — that's the "hard way" and is a common source of first-timer frustration (build tool mismatches, missing headers). Using the pre-built <code>pgvector/pgvector</code> image skips that entirely.
container_name: copilot-postgres	Without this, Docker would auto-generate a random name for the running container (like <code>enterprise-copilot-postgres-1</code>). We're naming it explicitly so that later commands (like the <code>docker exec</code> one below) can refer to it by a name we chose and remember.
restart: unless-stopped	If the container crashes or your machine reboots, Docker will automatically start it again — <i>unless</i> you manually stopped it yourself. Convenient for local dev so you don't have to remember to restart Postgres every time you reboot.
environment:	Environment variables passed into the container at startup. The Postgres image is specifically written to look for these three variable names and use them to initialize itself on first run.
POSTGRES_USER: copilot	The username of the database superuser that gets created automatically the first time the container starts.
POSTGRES_PASSWORD: copilot_dev_pw	That user's password. (This is a throwaway local-dev password, fine to hardcode here — never do this for anything that touches the real internet.)
POSTGRES_DB: copilot_db	The name of the database to auto-create on first startup, so you're not left with just the default <code>postgres</code> database.

Line	What it means
<code>ports: - "5432:5432"</code>	Maps a port on <i>your</i> machine (the number on the left) to a port <i>inside the container</i> (the number on the right). Postgres always listens on 5432 inside its container; this line makes that same port reachable from your own terminal/scripts as <code>localhost:5432</code> . Without this line, the container would run, but nothing outside Docker could connect to it.
<code>volumes: (under the service)</code>	Two entries, doing two different jobs — see next two rows.
<code>- pgdata:/var/lib/postgresql/data</code>	This is the important one for not losing your data. Containers are normally <i>disposable</i> — if you delete and recreate one, everything inside it is gone. <code>/var/lib/postgresql/data</code> is the folder <i>inside</i> the container where Postgres actually stores your tables' data on disk. This line maps that folder to a named volume called <code>pgdata</code> , which Docker manages <i>outside</i> the container's disposable filesystem — so your data survives even if you delete and recreate the <code>postgres</code> container.
<code>- ./db/init.sql:/docker- entrypoint-initdb.d/init.sql</code>	A second, different kind of volume mapping: this takes the <code>init.sql</code> file sitting in <i>your</i> project folder (<code>./db/init.sql</code>) and makes it appear inside the container at a specific path the Postgres image is specifically watching (<code>/docker-entrypoint-initdb.d/</code>) — any <code>.sql</code> file placed there is automatically run once, the very first time the container initializes its database. This is how our schema (tables, extension, index) gets created without you typing SQL by hand.
<code>healthcheck:</code>	Tells Docker how to check "is this database actually ready to accept queries yet," not just "has the container process started." Postgres can take a moment after starting before it's ready to serve traffic.
<code>test: ["CMD-SHELL", "pg_isready -U copilot -d copilot_db"]</code>	The actual check command. <code>pg_isready</code> is a small utility bundled with Postgres whose only job is to answer "yes/no, is this database accepting connections" — much more reliable than just checking if the container process is running.
<code>interval: 5s / timeout: 3s / retries: 10</code>	Run that check every 5 seconds, give each check up to 3 seconds to respond, and try up to 10 times before giving up and marking the container "unhealthy."
<code>volumes: (bottom-level, outside services)</code>	Declares <code>pgdata</code> as a named volume Docker should manage — this is what the <code>pgdata:/var/lib/postgresql/data</code> line above refers to. It has to be declared here once at the file's top level, separately from being <i>used</i> inside the service.

□ Concept primer: what is a Postgres "extension"?

Postgres ships with a core set of data types and features (integers, text, dates, etc.) but is deliberately designed to be extendable — an **extension** is a plugin that adds new capabilities (new data types, new functions, new index types) directly into the database engine. `pgvector` is one such extension: it adds a new `vector` data type and new distance-calculation operators (`<->`, `<=>`, `<#>`) that don't exist in stock Postgres. The image we chose already has the extension's code physically present on disk — but it still needs to be explicitly turned on *per-database* with a SQL command, which is the very first line of `init.sql` below.

`db/init.sql` — line by line

```

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS documents (
    id BIGSERIAL PRIMARY KEY,
    source TEXT NOT NULL,
    content TEXT NOT NULL,
    embedding vector(1536) NOT NULL,
    metadata JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX IF NOT EXISTS documents_embedding_hnsw_idx
    ON documents
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 64);

CREATE TABLE IF NOT EXISTS incidents (
    id BIGSERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    severity TEXT CHECK (severity IN ('low', 'medium', 'high', 'critical')),
    status TEXT DEFAULT 'open',
    created_at TIMESTAMPTZ DEFAULT now()
);

```

Line

What it means

CREATE EXTENSION IF NOT EXISTS vector;

Turns on the pgvector extension *for this specific database* (`copilot_db`). IF NOT EXISTS makes this command safe to re-run — without it, running the script twice would error out on "extension already exists" instead of silently doing nothing.

CREATE TABLE IF NOT EXISTS documents (...)

Creates our RAG knowledge-base table, only if it doesn't already exist (same safety reasoning as above).

id BIGSERIAL PRIMARY KEY

BIGSERIAL is Postgres shorthand for "a big auto-incrementing integer" (1, 2, 3... assigned automatically) — you never set this yourself. PRIMARY KEY marks it as the unique identifier for each row.

source TEXT NOT NULL

A plain text column (which document this chunk came from), and NOT NULL means Postgres will reject any insert that leaves this blank.

content TEXT NOT NULL

The actual chunk of text we'll embed and later retrieve.

embedding vector(1536) NOT NULL

This is the new type pgvector added. `vector(1536)` means "a vector with exactly 1536 numbers in it" — 1536 because that's the fixed output size of the specific embedding model (`text-embedding-3-small`) we're planning to use; every embedding model has its own fixed dimension, and this number **must** match whatever model you actually call, or inserts will fail.

metadata JSONB DEFAULT '{}'

A flexible column for extra structured info we might want later (e.g., `{"author": "..."}`) without needing a schema migration every time we think of a new field. JSONB is Postgres's binary-optimized JSON type (faster to query than plain JSON). DEFAULT '{}' means if you don't provide one, it starts as an empty object.

created_at TIMESTAMPTZ DEFAULT now()

Automatically stamps each row with its creation time (`TIMESTAMPTZ` = timestamp with timezone, the safe default) unless you override it.

CREATE INDEX IF NOT EXISTS documents_embedding_hnsw_idx ON documents

Creates an index (named `documents_embedding_hnsw_idx`) on the `documents` table. An index is a separate data structure that lets the database find rows fast without scanning every single one — same idea as a book's index letting you skip straight to a page instead of reading cover to cover.

Line	What it means
<pre> USING hns w (embedding vector_cosine_ops) </pre>	Specifies <i>which kind</i> of index (<code>hns w</code> — a graph-based structure built specifically for fast approximate nearest-neighbor search over vectors) and <i>which column and comparison method</i> it should optimize for (<code>embedding</code>, compared using cosine similarity via <code>vector_cosine_ops</code>). Without specifying <code>vector_cosine_ops</code>, the index would default to a different distance metric that might not match the operator (<code><=></code>) you actually query with.
<pre> WITH (m = 16, ef_construction = 64) </pre>	Two tuning knobs for the HNSW graph's shape/quality-vs-build-time trade-off — full explanation in the ADR in Section 6, since it's genuinely a "why this number" question worth understanding deeply rather than just copying.
<pre> CREATE TABLE IF NOT EXISTS incidents (...) </pre>	A completely separate, simple table — no vectors involved — that our <i>action-side</i> MCP tools will read/write later (Section 5). <code>severity TEXT CHECK (severity IN (...))</code> is Postgres enforcing, at the database level, that this column can only ever contain one of those four exact strings — an insert with <code>severity = 'urgent'</code> would be rejected outright.

Bringing the container up — every command explained

```

docker compose up -d
docker compose ps
docker exec -it copilot-postgres psql -U copilot -d copilot_db -c "\dx"

```

`docker compose up -d`

- `docker compose` — the subcommand that reads your `docker-compose.yml` and acts on it.
- `up` — start the service(s) defined in the file (downloading the image first if you don't already have it locally).
- `-d` — short for "detached": run the container in the background and give you your terminal prompt back immediately. **Alternative:** omit `-d` and the container runs in the foreground, printing its logs directly into your terminal and blocking it until you `Ctrl+C` — useful the very first time you run it, so you can watch for errors, but inconvenient for everyday use.

`docker compose ps`

- Lists the containers this compose file manages and their current status. You're watching for the `STATUS` column to go from `starting` to `healthy` (that word "healthy" is coming directly from the `healthcheck` block we wrote above) before trying to connect to it.

```

docker exec -it copilot-postgres psql -U copilot -d copilot_db -c "\dx"

```

This single line is actually two commands chained together — Docker's and Postgres's own client tool. Broken into every piece:

- `docker exec` — "run a new command *inside* an already-running container" (as opposed to `docker run`, which would start a brand-new container).
- `-it` — actually two flags combined: `-i` (interactive — keep input open) and `-t` (allocate a terminal) — together they make the command behave like a normal interactive terminal session instead of firing-and-forgetting. Needed here mainly by convention for `psql`; for the specific one-shot `-c` usage below it's not strictly required, but doesn't hurt.
- `copilot-postgres` — *which* running container to run the command inside — this is the `container_name` we chose in the compose file. If we hadn't set that name explicitly, we'd have to look up Docker's auto-generated name first.
- `psql` — the actual command being run *inside* that container. `psql` is Postgres's own official command-line client, and it's already installed inside the `pgvector/pgvector` image (it ships with every Postgres install) — we're not installing anything new, just invoking a tool that's already there.
- `-U copilot` — connect as the user `copilot` (matches `POSTGRES_USER` from the compose file).
- `-d copilot_db` — connect to the database named `copilot_db` (matches `POSTGRES_DB`).
- `-c "\dx"` — `-c` means "run this one command non-interactively and exit" (instead of dropping you into `psql`'s interactive prompt). `\dx` is a **psql meta-command** (all of `psql`'s built-in meta-commands start with a backslash, to distinguish them from actual SQL) that lists every extension currently installed in the connected database — this is how you visually confirm `vector` shows up in that list, proving `CREATE EXTENSION` from `init.sql` actually ran successfully.
- **Alternative:** you could instead run `docker exec -it copilot-postgres psql -U copilot -d copilot_db without -c`, which drops you into a live interactive `psql>` prompt where you can type `\dx`, then SQL queries, then `\q` to quit. That's actually the more common

way people use `psql` day-to-day once comfortable with it — we used the one-shot `-c` form here purely so it's a single copy-pasteable verification line.

□ Concept primer: what does the sanity-check script actually verify, and why Node.js/TypeScript instead of just more SQL?

We already confirmed the extension is installed via `psql`. The sanity script exists to confirm a *different* thing: that **your application code**, written in the same language and using the same database driver library you'll use for the real app, can successfully connect, insert a vector, and run a similarity query — end to end, exactly the way your real backend eventually will. It's a smaller, faster version of "does my whole stack talk to itself correctly" before you build anything more complicated on top of it.

`pg` is the standard, most widely-used Node.js library ("driver") for talking to Postgres — it does not need to know anything special about `pgvector`; as far as `pg` is concerned, a vector is just a specially-formatted string it sends to Postgres, which is why we `JSON.stringify()` the array before sending it.

scripts/sanity-check-pgvector.ts — line by line

```
import { Client } from "pg";

async function main() {
  const client = new Client({
    host: "localhost",
    port: 5432,
    user: "copilot",
    password: "copilot_dev_pw",
    database: "copilot_db",
  });
  await client.connect();

  const fakeEmbedding = new Array(1536).fill(0);
  fakeEmbedding[0] = 1;

  await client.query(
    `INSERT INTO documents (source, content, embedding) VALUES ($1, $2, $3)`,
    ["sanity-check", "hello world", JSON.stringify(fakeEmbedding)]
  );

  const res = await client.query(
    `SELECT id, source, embedding <=> $1 AS distance
    FROM documents
    ORDER BY embedding <=> $1
    LIMIT 3`,
    [JSON.stringify(fakeEmbedding)]
  );

  console.log("Nearest neighbors:", res.rows);
  await client.end();
}

main().catch(console.error);
```

Line	What it means
`import { Client } from "pg";`	Pulls in the `Client` class from the `pg` library — this is the object representing one connection to a Postgres database.
`async function main() { ... }`	We wrap everything in a function marked `async` because every database call is asynchronous (it takes real time, over the network, and JavaScript doesn't want to freeze your whole program waiting

for it) — `async/await` is how modern JS/TS lets you write asynchronous code that still *reads* top-to-bottom like normal synchronous code. | `const client = new Client({...})` | Creates a new connection object, configured with exactly the same host/port/user/password/database values we set up in the compose file — this is the connection *recipe*, nothing has connected yet at this point. | `host: "localhost", port: 5432` | Because of the `ports: "5432:5432"` mapping in the compose file, the container is reachable from your own machine at `localhost:5432` as if Postgres were installed natively. | `await client.connect()` | Actually opens the network connection. `await` pauses this function here until the connection succeeds (or throws an error if it can't connect — e.g., if the container isn't running yet). | `const fakeEmbedding = new Array(1536).fill(0)` | Creates a plain JavaScript array of 1536 zeros — matching the `vector(1536)` column size exactly. This is a placeholder standing in for what would normally come from calling a real embedding model. | `fakeEmbedding[0] = 1` | Sets just the first number to 1, so this fake vector isn't *all* zeros (an all-zero vector is a mathematically degenerate edge case for cosine distance — undefined/meaningless — so we avoid it even in a throwaway test). | `await client.query('INSERT INTO...', [...])` | Runs a parameterized SQL insert. The `$1, $2, $3` placeholders in the SQL string get safely filled in from the array passed as the second argument, in order — *this is the standard, safe way to insert dynamic values into SQL*, because it prevents SQL injection (never build a query by directly concatenating strings together). | `JSON.stringify(fakeEmbedding)` | Converts the JS array into a text string like `"[0,1,0,0,...]"` — this is the text format `pgvector` expects to receive for a `vector` column over the wire; the `pgdriver` doesn't need any special vector-aware code because it's really just sending text. | `const res = await client.query('SELECT...', [...])` | Runs the similarity search — `embedding <=> $1` computes cosine distance between every row's stored embedding and our fake query vector; `ORDER BY` that same expression sorts nearest-first; `LIMIT 3` caps it at 3 results. | `res.rows` | The `pg` library returns query results as an object with a `rows` array — each element is one result row as a plain JS object (e.g., `{id: 1, source: "sanity-check", distance: 0}`). | `console.log("Nearest neighbors:", res.rows)` | Prints the results so you can visually inspect them. | `await client.end()` | Closes the connection cleanly — without this, the script's Node process might hang after finishing, since an open database connection is a live network resource. | `main().catch(console.error)` | Calls our async function, and if it throws anywhere (bad password, container not running, syntax error), `.catch(console.error)` prints the error instead of letting it silently disappear as an "unhandled promise rejection." |

Run with `npx tsx scripts/sanity-check-pgvector.ts`. `npx` runs a locally-installed CLI tool without needing it globally installed system-wide; `tsx` is a small tool that lets Node run a `.ts` file directly, compiling TypeScript to JavaScript on the fly, so you don't need a separate manual build step for a one-off script. If this prints a row with `distance: 0` for the row you just inserted (a vector has zero distance from itself), your extension, index, and driver plumbing all work end to end.

2.2 Playwright setup

```
npm init playwright@latest -- --yes --quiet
# Choose: TypeScript, tests folder = e2e, GitHub Actions = yes, install browsers = yes
```

playwright.config.ts:

```
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  testDir: "./e2e",
  timeout: 30_000,
  expect: { timeout: 10_000 }, // generous - LLM streaming is slower than typical UI actions
  fullyParallel: true,
  retries: process.env.CI ? 2 : 0,
  reporter: [
    ["html", { open: "never" }],
  ],
  use: {
    baseURL: "http://localhost:5173",
    trace: "on-first-retry",
    screenshot: "only-on-failure",
  },
  webServer: {
    command: "npm run dev",
    url: "http://localhost:5173",
    reuseExistingServer: !process.env.CI,
  },
  projects: [
    { name: "chromium", use: { ...devices["Desktop Chrome"] } },
  ],
});
```

Run modes:

```
npx playwright test           # headless, CI mode
npx playwright test --ui      # interactive UI mode - best while writing tests
npx playwright test --debug   # step-through debugger
npx playwright show-report     # view last HTML report
```

3. Tech Stack & Package Manifest

Root (package.json, workspaces):

```
npm workspaces: ["backend", "frontend"]
devDependencies: typescript, tsx, @types/node, playwright, @playwright/test, eslint, prettier
```

Backend (backend/package.json):

```
dependencies:
  fastify, @fastify/cors, @fastify/sse (or manual SSE via reply.raw)
  @langchain/core, @langchain/langgraph, @langchain/openai
  @modelcontextprotocol/sdk
  pg, pgvector (npm helper, optional - raw SQL works fine)
  zod (schema validation for MCP tool args + LangGraph state)
  dotenv
devDependencies:
  typescript, tsx, @types/node, @types/pg
```

Frontend (frontend/package.json):

```
dependencies:
  react, react-dom
  @microsoft/fetch-event-source (robust SSE client - plain EventSource can't do POST)
devDependencies:
  typescript, vite, @vitejs/plugin-react, @types/react, @types/react-dom
```

4. Repository & File Structure

```
enterprise-copilot/
├─ docker-compose.yml
├─ db/
│   └─ init.sql
├─ scripts/
│   ├── sanity-check-pgvector.ts
│   └─ ingest-docs.ts # [Code By Hand]
├─ backend/
│   ├── package.json
│   ├── tsconfig.json
│   └─ src/
│       ├── server.ts # [Code By Hand] Fastify app + SSE route
│       ├── db/
│       │   └─ pool.ts # [Code By Hand] pg Pool singleton
│       ├── rag/
│       │   ├── embeddings.ts # [Code By Hand] embedding client wrapper
│       │   └─ retriever.ts # [Code By Hand] similarity_search()
│       ├── mcp/
│       │   ├── server.ts # [Code By Hand] MCP tool server
│       │   └─ client.ts # [Code By Hand] MCP client used by the graph
│       └─ agent/
│           ├── state.ts # [Code By Hand] LangGraph state schema
│           ├── nodes.ts # [Code By Hand] classify/retrieve/act/answer nodes
│           └─ graph.ts # [Code By Hand] StateGraph wiring + edges
├─ frontend/
│   ├── package.json
│   ├── vite.config.ts
│   └─ src/
│       ├── main.tsx # [AI Scaffold]
│       ├── App.tsx # [AI Scaffold]
│       ├── components/
│       │   ├── ChatWindow.tsx # [AI Scaffold]
│       │   ├── ToolCallCard.tsx # [AI Scaffold]
│       │   └─ StreamingMessage.tsx # [AI Scaffold]
│       └─ hooks/
│           └─ useChatStream.ts # [Code By Hand] - the data contract matters; don't outsource this
└─ e2e/
    ├── chat-stream.spec.ts # [Code By Hand]
    └─ fixtures/
        └─ mock-llm-server.ts # [Code By Hand]
```

The one deliberate exception to "UI is AI-scaffolded": `useChatStream.ts`. This is the hook that parses your SSE event contract (`token`, `tool_call`, `tool_result`, `done`, `error`) into React state. It's not visual design — it's the client half of your streaming data contract, and if you don't understand it cold you can't debug race conditions or write meaningful Playwright assertions against it. Everything downstream of it (how it *looks*) can be scaffolded freely.

5. Code Breakdown by File

backend/src/agent/state.ts — [Code By Hand]

Purpose: Defines the shared state object that flows through every LangGraph node — the single source of truth for "what does the agent currently know."

```
import { Annotation } from "@langchain/langgraph";

export const AgentState = Annotation.Root({
  messages: Annotation<{ role: string; content: string }[]>({
    reducer: (prev, next) => prev.concat(next),
    default: () => [],
  }),
  intent: Annotation<"retrieve" | "act" | "both" | "chitchat">({
    reducer: (_prev, next) => next,
    default: () => "chitchat",
  }),
  retrievedChunks: Annotation<{ content: string; source: string; score: number }[]>({
    reducer: (_prev, next) => next,
    default: () => [],
  }),
  toolResults: Annotation<{ tool: string; result: unknown }[]>({
    reducer: (prev, next) => prev.concat(next),
    default: () => [],
  }),
  loopCount: Annotation<number>({
    reducer: (_prev, next) => next,
    default: () => 0,
  }),
});
```

2-3 sentence explanation: LangGraph's `Annotation` API lets each state field declare its own **reducer** — how new node output merges with existing state, rather than a blind overwrite. `messages` and `toolResults` accumulate (`concat`), while `intent` and `retrievedChunks` are fully replaced each time they're set. `loopCount` exists purely as a cycle breaker (see ADR #3 in Section 6) — without it, a mis-classified "act" loop can spin forever.

backend/src/agent/nodes.ts — [Code By Hand]

Purpose: The actual node functions — the "brains" the graph wires together.

```

async function classifyIntent(state: typeof AgentState.State): Promise<Partial<typeof AgentState.State>> {
  // Cheap classification call (small/fast model) to route between retrieve/act/both/chitchat
}

async function retrieveNode(state: typeof AgentState.State): Promise<Partial<typeof AgentState.State>> {
  // Calls rag/retriever.ts similarity_search, attaches top-k chunks to state
}

async function actNode(state: typeof AgentState.State): Promise<Partial<typeof AgentState.State>> {
  // Calls mcp/client.ts to invoke a tool (e.g. create_incident_ticket), appends result
}

async function synthesizeAnswer(state: typeof AgentState.State): Promise<Partial<typeof AgentState.State>> {
  // Final LLM call combining messages + retrievedChunks + toolResults into a streamed answer
}

```

Each node is a pure-ish async function: `(state) => partial state update`. This is the core mental model shift from a single-prompt chain — you're not writing "one big prompt," you're writing small, independently testable functions that each read a slice of state and return a delta.

backend/src/agent/graph.ts — [Code By Hand]

Purpose: Wires nodes into a `StateGraph` with conditional edges — the actual "decision-making" structure.

```

function buildGraph() {
  const graph = new StateGraph(AgentState)
    .addNode("classify", classifyIntent)
    .addNode("retrieve", retrieveNode)
    .addNode("act", actNode)
    .addNode("answer", synthesizeAnswer)
    .addEdge(START, "classify")
    .addConditionalEdges("classify", (state) => {
      if (state.intent === "retrieve") return "retrieve";
      if (state.intent === "act") return "act";
      if (state.intent === "both") return "retrieve"; // retrieve first, then act
      return "answer"; // chitchat
    })
    .addConditionalEdges("retrieve", (state) =>
      state.intent === "both" ? "act" : "answer"
    )
    .addEdge("act", "answer")
    .addEdge("answer", END);

  return graph.compile();
}

```

`addConditionalEdges` is the mechanism that makes this a *graph* rather than a linear chain — the routing function inspects state and picks the next node at runtime. This is precisely the "why LangGraph over a plain LangChain `AgentExecutor`" distinction interviewers probe for: LangGraph makes control flow explicit and inspectable (and resumable/checkpointable) instead of implicit inside a ReAct loop's prompt.

backend/src/rag/retriever.ts — [Code By Hand]

Purpose: The actual pgvector similarity query.

```

async function similaritySearch(
  queryEmbedding: number[],
  k: number = 5
): Promise<{ content: string; source: string; score: number }[]> {
  const result = await pool.query(
    `SELECT content, source, 1 - (embedding <=> $1) AS score
    FROM documents
    ORDER BY embedding <=> $1
    LIMIT $2`,
    [JSON.stringify(queryEmbedding), k]
  );
  return result.rows;
}

```

1 - cosine_distance converts distance into a similarity score (1.0 = identical) purely for readability downstream; the ORDER BY ... LIMIT pattern is what lets the HNSW index kick in instead of a full sequential scan — Postgres's query planner recognizes this shape against an indexed vector column.

backend/src/mcp/server.ts — [Code By Hand]

Purpose: Exposes tools (search_incidents, create_incident_ticket, notify_channel) via the MCP protocol so the agent (or any MCP-compatible client) can call them generically.

```

const server = new McpServer({ name: "copilot-tools", version: "1.0.0" });

server.tool(
  "create_incident_ticket",
  "Create a new incident ticket",
  { title: z.string(), severity: z.enum(["low", "medium", "high", "critical"]) },
  async ({ title, severity }) => {
    const row = await pool.query(
      `INSERT INTO incidents (title, severity) VALUES ($1, $2) RETURNING id`,
      [title, severity]
    );
    return { content: [{ type: "text", text: `Created incident #${row.rows[0].id}` }] };
  }
);

```

This is the crux of "why MCP instead of a custom REST tool-calling layer" (ADR #2): the tool's schema (via zod) is declared once and is introspectable by *any* MCP client, not just this one agent — the same server could be wired into Claude Desktop or another agent without rewriting the integration.

frontend/src/hooks/useChatStream.ts — [Code By Hand]

Purpose: Parses the SSE event stream into React state — messages, pendingToolCalls, isStreaming.

```

function useChatStream(threadId: string) {
  // Uses @microsoft/fetch-event-source to POST + stream
  // Reduces incoming events (token/tool_call/tool_result/done/error) into local state
}

```

Signature: `useChatStream(threadId: string): { messages: Message[]; sendMessage: (text: string) => void; isStreaming: boolean }`. The subtlety worth understanding by hand: tokens must append to the *last* assistant message, but a `tool_call` event should insert a new distinct UI element rather than being treated as text — getting this reducer wrong is the #1 source of the "streaming UI race condition" battle scar in Section 6.

e2e/chat-stream.spec.ts — [Code By Hand]

Covered fully in Section 7.

6. Architectural Reasoning You Should Be Able to Defend

Framed as decisions and trade-offs you now understand deeply enough to argue either side of — the honest version of "speaking the language."

Why this stack over the obvious alternatives

- **A single LangChain AgentExecutor (ReAct loop) instead of LangGraph** would work for a demo, but control flow lives implicitly inside a prompt loop — you can't easily inspect "why did it choose to retrieve," checkpoint mid-execution, or bound the number of tool-call cycles. LangGraph's explicit `StateGraph` makes routing a first-class, testable thing.
- **Custom REST endpoints per tool instead of MCP** means every new agent or client that wants to call `create_incident_ticket` needs its own bespoke integration and its own understanding of the tool's shape. MCP standardizes tool discovery and invocation so the same server works across agents/clients without rewriting glue code — this is the same "USB-C for AI tools" pitch you'll hear repeated, and it's a legitimate one once you've built a server and seen a second client just... work against it.
- **A flat vector table with brute-force scan instead of HNSW indexing** is fine at hundreds of documents; it falls over past tens of thousands, because cosine distance against every row is $O(n)$ per query.

Three trade-offs worth being able to argue in depth

1. HNSW vs. IVFFlat for the vector index. IVFFlat requires choosing a number of "lists" up front and needs the table already populated (and periodically re-clustered) to build a good index — it degrades if data drifts after index creation. HNSW builds an incrementally-updatable graph structure, has no training step, and generally gives better recall at similar speed for datasets that grow over time — at the cost of higher memory usage and slower index *build* time. For a knowledge base that's continuously ingesting new docs (not a static batch), HNSW's incremental nature is the deciding factor over IVFFlat's need for re-training.

2. Where to put the "cycle breaker" in the agent graph. Once the graph has any conditional loop (e.g., "act, then check if the action needs a follow-up act"), an unbounded loop is a real production risk — a misclassification or a flaky tool call can spin the graph indefinitely, burning tokens and latency budget. The trade-off is *where* to bound it: a global `loopCount` in shared state (simple, but blunt — kills all cycles uniformly) vs. per-node retry budgets (more precise, more state to manage). For a first production version, a global counter with a hard ceiling (e.g., 5) plus a fallback "answer with what you have" node is the right complexity-to-safety ratio; per-node budgets are worth it once you have real telemetry showing which node actually loops.

3. Streaming granularity: token-level vs. message-level SSE events. Token-level streaming feels more responsive but multiplies event volume and makes the frontend reducer logic more fragile (see the race condition below). Message-level (stream whole sentences or whole tool results) is simpler to reason about and test, at the cost of perceived latency. The practical answer: stream tokens for the final synthesized answer (where perceived responsiveness matters most), but treat tool calls and their results as atomic, single events — never streamed token-by-token, since a half-finished tool-call JSON payload is useless to the UI anyway.

Three non-obvious failure modes and how to reason about them

- 1. Streaming UI race condition:** if `tool_call` and `token` events arrive close together, a naive reducer that always appends to "the last message" can attach a tool result's text to the wrong bubble, or interleave partial tokens from two different assistant turns if the user sends a

second message before the first stream closes. Fix: tag every event with the `threadId` *and* a monotonic `turnId`, and have the reducer ignore/queue events whose `turnId` doesn't match the currently-rendering turn.

2. **Context window overflow from unbounded retrieved-chunk accumulation:** if `retrievedChunks` in state isn't capped or summarized across a multi-turn conversation, a long session can silently exceed the model's context window several turns in, causing a hard-to-reproduce failure that only shows up after "it's been working fine for a while." Fix: cap `k` per retrieval and explicitly truncate/summarize `messages` history rather than letting it grow unbounded — this is the same class of bug as the classic $n+1$ query problem, just in tokens instead of DB calls.
3. **Playwright flakiness against real LLM latency:** naive `page.waitForTimeout(2000)` calls become either too short (flaky failures) or too long (slow suites) because LLM response time is inherently variable. Fix: never wait on fixed timers — use Playwright's web-first assertions (`expect(locator).toHaveText(...)`, `expect(locator).toBeVisible()`) which auto-retry against real DOM state until it matches or the assertion timeout is hit, and for CI, mock the LLM layer entirely (see Section 7) so test runtime doesn't depend on model latency at all.

7. Playwright Test Strategy (E2E & Agent Testing)

Mocking vs. live LLM — the actual decision rule

Run against a **mocked backend/LLM layer in CI** (fast, deterministic, free), and reserve a small number of **live-LLM smoke tests** for local/manual runs or a nightly job (catches real integration drift — prompt changes, SDK upgrades — that mocks can't). Never gate a PR merge on live-LLM test output; non-determinism there is a feature of the model, not a bug in your test.

The mock lives at the SSE-endpoint level, not deep inside LangGraph — you stand up a tiny Fastify server on a test port that emits a scripted sequence of SSE events (`token`, `tool_call`, `tool_result`, `done`) with real timing gaps, and point the frontend at it. This tests your actual frontend contract (`useChatStream.ts`) end-to-end without touching a real model.

e2e/fixtures/mock-llm-server.ts:

```
export function startMockStreamServer(port: number, events: SseEvent[]) {
  // Fastify server: on POST /chat/stream, writes each event in `events`
  // to the response with a small artificial delay, then closes the stream.
}
```

Sample spec — web-first assertions on streaming UI

```

import { test, expect } from "@playwright/test";
import { startMockStreamServer } from "../fixtures/mock-llm-server";

test.describe("chat streaming", () => {
  test("renders streamed tokens and a tool-call card in order", async ({ page }) => {
    const events = [
      { type: "token", data: "Checking " },
      { type: "token", data: "open incidents..." },
      { type: "tool_call", data: { tool: "search_incidents", args: {} } },
      { type: "tool_result", data: { tool: "search_incidents", result: "3 open" } },
      { type: "token", data: " Found 3 open incidents." },
      { type: "done", data: {} },
    ];

    await startMockStreamServer(4001, events);

    await page.goto("/");
    await page.getByRole("textbox", { name: "message" }).fill("any open incidents?");
    await page.getByRole("button", { name: "send" }).click();

    // Web-first assertion: auto-retries until the streamed text is fully rendered
    await expect(page.getByTestId("assistant-message")).toHaveText(
      "Checking open incidents... Found 3 open incidents.",
      { timeout: 10_000 }
    );

    // Tool call card should appear between the two token chunks, not after
    await expect(page.getByTestId("tool-call-search_incidents")).toBeVisible();
  });
});

```

Note what's absent: no `waitForTimeout`, no manual sleeps. `toHaveText` polls the DOM until it matches or times out — this is what "web-first assertion" means in Playwright's own terminology, and it's the single most important habit for testing anything async.

8. Fresher-to-IC Onboarding Plan (15 sessions, ~2–2.5 hrs each)

This replaces a flat hour-by-hour syllabus with an actual **mentorship ramp**. The hand-holding level is explicit per session and decreases on a curve, not a cliff — by Week 3 you're getting specs and code review, not code.

Every session follows the same four-beat structure, same as pairing a fresher with a senior dev:

- ☐ **Study first** — a specific, narrow thing to read/watch *before* touching the keyboard, with a concrete question to answer from it (not "read the docs," but "read X and be able to answer Y").
- ☐ **Guided build** — the hands-on work, with the amount of code I hand you shrinking every session (see the scale below).
- ☐ **Solo stretch** — a small task you do completely unaided, scoped to something the guided build just taught.
- ☐ **Checkpoint** — one question you should be able to answer out loud before moving on. If you can't, redo the solo stretch before the next session, don't push forward.

Hand-holding scale across the 3 weeks:

- Week 1 (Sessions 1–5): I give you ~80% complete code with line-by-line rationale; your solo stretch is a small, safe modification (change a parameter, add a field).

- Week 2 (Sessions 6–10): I give you function signatures + the tricky 20% (the part with a non-obvious gotcha) and describe the rest in prose; you write the routine parts yourself.
- Week 3 (Sessions 11–15): I give you a spec and an interface contract only ("this function must accept X, return Y, handle Z edge case"); you write the whole thing, then walk me through it as if in a design review.

Week 1 — pgvector + RAG (heavy pairing)

Session 1 — Environment & first vector query

- ☐ Study: pgvector's three distance operators (`<->`, `<=>`, `<#>`). Be able to say which one you'd use for embeddings normalized to unit length and why.
- ☐ Guided: I walk you through `docker-compose.yml` + `init.sql` line by line before you run anything; you type the commands, I explain each one as it runs.
- ☐ Solo: change the HNSW `m` parameter from 16 to 32, rebuild the index, and note whether/why build time changes.
- ☐ Checkpoint: "Why does `<=>` need vectors of the same dimension, and what happens if they aren't?"

Session 2 — Embeddings & ingestion

- ☐ Study: what an embedding actually is (a fixed-length float vector encoding semantic meaning) — skim one short explainer, no math required.
- ☐ Guided: I give you a mostly-complete `ingest-docs.ts`; you fill in the chunking loop (splitting a doc into ~500-token pieces) — a small, bounded gap.
- ☐ Solo: write 3 more fake runbook docs and ingest them yourself, no help.
- ☐ Checkpoint: "Why chunk documents instead of embedding the whole file as one vector?"

Session 3 — Similarity search, by hand

- ☐ Study: none new — re-read your own `init.sql` from Session 1 and predict what `ORDER BY embedding <=> $1 LIMIT 5` will return before running it.
- ☐ Guided: you write `retriever.ts` yourself from a skeleton I give you (function signature + SQL template with one blank to fill); I review it live.
- ☐ Solo: run 3 different test queries against your data, including one you expect to return nothing good — write down what actually came back.
- ☐ Checkpoint: "What does a similarity score of 0.4 vs 0.9 practically mean for whether you should trust the retrieved chunk?"

Session 4 — Seeing the index matter

- ☐ Study: what `EXPLAIN ANALYZE` output means at a glance (seq scan vs. index scan lines).
- ☐ Guided: together, duplicate your dataset a few hundred times and run `EXPLAIN ANALYZE` before and after the HNSW index exists; I narrate what changed and why.
- ☐ Solo: drop the index, re-run, and describe the difference in your own words (no help from me this time).
- ☐ Checkpoint: "At roughly what data size did brute-force scanning start to feel slow, and why would that number be different for a 3072-dim embedding?"

Session 5 — Consolidate + reflect

- ☐ Study: none — this session is synthesis.
- ☐ Guided: you narrate the full RAG path (chunk → embed → store → query) to me from memory; I only interject where you're fuzzy.
- ☐ Solo: write 3-4 sentences, unprompted, on the one thing about RAG that surprised you this week. This is your first honestly-earned "battle scar" note.
- ☐ Checkpoint: none — this is the review session.

Week 2 — MCP + LangGraph (collaborative, 50/50)

Session 6 — Your first MCP tool

- ☐ Study: the MCP spec's core idea in one paragraph — tools as introspectable, schema-declared functions any client can call.
- ☐ Guided: I give you the `zod` schema and the tool *name/description*; you write the handler body (the SQL insert) yourself.

- ☐ Solo: add a second tool (`search_incidents`) completely on your own using the first as a template.
- ☐ Checkpoint: "What would break for a second agent trying to use your server if you hadn't declared the zod schema?"

Session 7 — MCP client + wiring

- ☐ Study: skim how a client discovers a server's tools (`tools/list`) before calling one (`tools/call`).
- ☐ Guided: I describe the client's job in prose; you write `mcp/client.ts` from that description with the SDK docs open.
- ☐ Solo: call your new `search_incidents` tool from a throwaway script, unaided.
- ☐ Checkpoint: "Why does the client need to call `tools/list` at all instead of just hardcoding the tool names it knows about?"

Session 8 — LangGraph state

- ☐ Study: LangGraph's `Annotation/reducer` concept — one worked example from the docs.
- ☐ Guided: I give you the field list for `AgentState`; you write the reducers yourself and justify each choice (concat vs. replace) to me.
- ☐ Solo: add a new state field (e.g., `errorCount`) with a reducer of your choosing.
- ☐ Checkpoint: "What would go wrong if `messages` used a replace reducer instead of concat?"

Session 9 — Nodes

- ☐ Study: none new — reread your own `state.ts`.
- ☐ Guided: I give you `classifyIntent`'s signature and the prompt to use; you write the function body. You write `retrieveNode` fully unaided (it's just wiring Week 1's retriever into a node).
- ☐ Solo: write `actNode` calling your MCP client, unaided, using `retrieveNode`'s shape as a template.
- ☐ Checkpoint: "Why is each node written as `(state) => partial update` instead of mutating state directly?"

Session 10 — Wiring the graph + the cycle breaker

- ☐ Study: what an unbounded agent loop actually costs in production (tokens + latency), one short read.
- ☐ Guided: together, wire `addConditionalEdges`; I ask you to predict the routing before we run it, then we deliberately misconfigure one edge to watch it loop, and you fix it.
- ☐ Solo: add the `loopCount` cycle breaker unaided.
- ☐ Checkpoint: "Where exactly would you add a per-node retry budget instead of a global counter, and when would that actually be worth the extra complexity?"

Week 3 — Streaming, frontend contract, testing (mostly independent)

Session 11 — Streaming API

- ☐ Study: SSE vs. WebSockets in one paragraph — why SSE is the simpler right choice for one-directional token streaming.
- ☐ Spec only: "Build a Fastify route that streams LangGraph's `stream()` output as SSE events tagged `token/tool_call/tool_result/done`." You build it; I review.
- ☐ Solo: add an `error` event type for when a node throws, unaided.
- ☐ Checkpoint: walk me through your route as if I were reviewing your PR.

Session 12 — Frontend scaffold + the one hand-written hook

- ☐ Study: none — use an AI scaffolding tool for the UI shell per the plan; that's expected here, not a gap.
- ☐ Spec only: "useChatStream must append tokens to the last assistant message but insert tool calls as distinct elements, keyed by `turnId` so a second in-flight message can't corrupt the first." You write it from that spec.
- ☐ Solo: reproduce the race condition described in Section 6 on purpose (remove the `turnId` check) and watch it actually happen.
- ☐ Checkpoint: "Explain the race condition and your fix as if a junior engineer just introduced the bug in a PR."

Session 13 — Playwright, for real

- ☐ Study: "web-first assertions" in Playwright's own docs — one example.
- ☐ Spec only: build the mock SSE server and one passing spec from Section 7 unaided; I review only.
- ☐ Solo: write a second spec covering the tool-call path, and one spec you expect to fail on purpose (to confirm your assertions are actually meaningful).

- ☐ Checkpoint: none — this is a review session, treat it like a real PR review.

Session 14 — Full system run-through

- ☐ End-to-end: run the whole thing, docker up through UI, across all four intent types, with nothing new to build — this session is entirely about noticing rough edges and fixing them yourself.
- ☐ Solo: fix at least one real bug you find unaided.
- ☐ Checkpoint: narrate the full request lifecycle diagram from memory, unaided.

Session 15 — Interview defense prep

- ☐ Mock design review: answer, out loud, in your own words: why HNSW over IVFFlat, why MCP over custom REST, how you bound agent loops, how you test non-deterministic streaming UIs, and what you'd do differently building this a second time.
- No solo task — this session is the payoff of the previous 14.

A note on positioning: the strongest honest framing for interviews is "I identified this gap and closed it with a deliberately realistic, end-to-end system in three weeks" — paired with fluent, specific answers to the trade-off questions above. That combination reads as more credible than a vague claim of 1-2 years' tenure, precisely because it's verifiable and the depth in your answers will speak for itself.

